第 01 章

项目全貌与学习闭环

从一个真实学习需求出发，分清资料、题库、学习业务与辅导 Agent 的职责

这一章完成后，你能够

- 用真实页面和源码画出“资料→题库→作答→复习”的学习闭环。
- 准确使用 Model、Chain、Agent、State、Context、Snapshot 与 Checkpoint 等名称。
- 根据一次请求选择阅读入口，并解释服务、存储和后台任务的分工。
- 在统一环境运行首个离线实验，判断一条验证结论实际证明了什么。

学习前提

Python 函数、类、字典与基本异步语法；了解 HTTP 请求与响应；项目已按学习准备说明安装默认 .venv

代码基准：72dea46 + 当前已完成的学习准备。Python 3.12；LangChain 1.4.3；LangGraph 1.2.12。

正文采用场景、名词、步骤、源码、实验和练习的教学顺序。截图注明数据来源；教学简化与生产实现分别标记。

本章目录

1.1 从真实需求认识项目 3

1.2 用界面定位四条主路径 4

1.3 命名先对齐：Model、Chain 与 Agent 6

1.4 Context、State 与三种保存位置 6

1.5 从目录到分层：函数为什么放在这里 7

1.6 服务协作与失败定位 9

1.7 首个可运行实验：只看路由和版本 10

1.8 学习规范与证据怎么读 11

1.9 用一个问题串起八章 11

1.10 综合练习与参考思路 12

1.11 源码阅读清单与下一步 13

1.1 从真实需求认识项目

设想你上传一份网络基础讲义，希望系统帮你整理知识点、生成题目、安排练习，并在答错之后给出有资料依据的解释。这个需求包含多个阶段：资料先变成可处理文本；候选题要通过校验和审核；作答需要保存事实；辅导回答需要取证。学习项目的第一步，是把这些阶段拆清楚。



图 1-1 真实学习概览。这里的掌握度和待复习数量来自现有业务记录，不能直接当作模型质量分数。

当前真实项目页面 / 现有业务数据 / 2026-10-07。学习概览：资料、题库、练习与复习入口

StudyAgent 是本项目的产品名称。页面中导入的“api-server接口能力梳理”是用户学习资料，它描述的部署接口属于资料内容。教材的 TCP 文本则是固定教学样本。两者都可用于观察资料处理，但不能把导入资料里的 GPU、Deployment 等能力说成 StudyAgent 自身已实现的功能。



图 1-2 学习闭环的业务顺序。辅导按需进入，不是每次作答都必须调用同一个 Agent。

如果你只想解释某一段资料，可以从知识库辅导入口开始；如果你要练习并更新复习计划，应进入学习流程。一次辅导建议不会自动修改成绩或复习安排。把“解释建议”和“业务提交”分开，才能理解为什么数据库中同时存在学习记录和辅导会话。

1.2 用界面定位四条主路径

路径	输入与输出	主要入口
资料处理	文件→原文片段、元数据、索引	materials 路由；ingestion 模块
题库生成	已处理材料→待审核候选题	question_generation 链与任务
学习业务	标准化答案→判分、掌握度、复习	StudyService 与确定性规则
学习辅导	问题与固定范围→带引用回复	TutorService 与 create_tutor_agent

表 1-1 四条路径的触发输入、业务结果与代码入口



图 1-3 真实材料处理卡片：资料已就绪，但目标 10 道的题目生成只通过 9 道。资料状态和出题结果属于不同阶段。

当前真实项目页面 / 现有业务数据 / 2026-10-07。材料已就绪与生成目标10/通过9/拒绝1的实际记录

这张卡片能同时说明两件事。第一，材料 ready 表示资料处理链路已到可用阶段；第二，题目仍可能被质量门禁拒绝。目标数量是用户期望，通过数量是处理结果，两者不能混写为“生成成功 10 道”。当前截图拒绝 1 道的原因是片段与引文数量不一致。

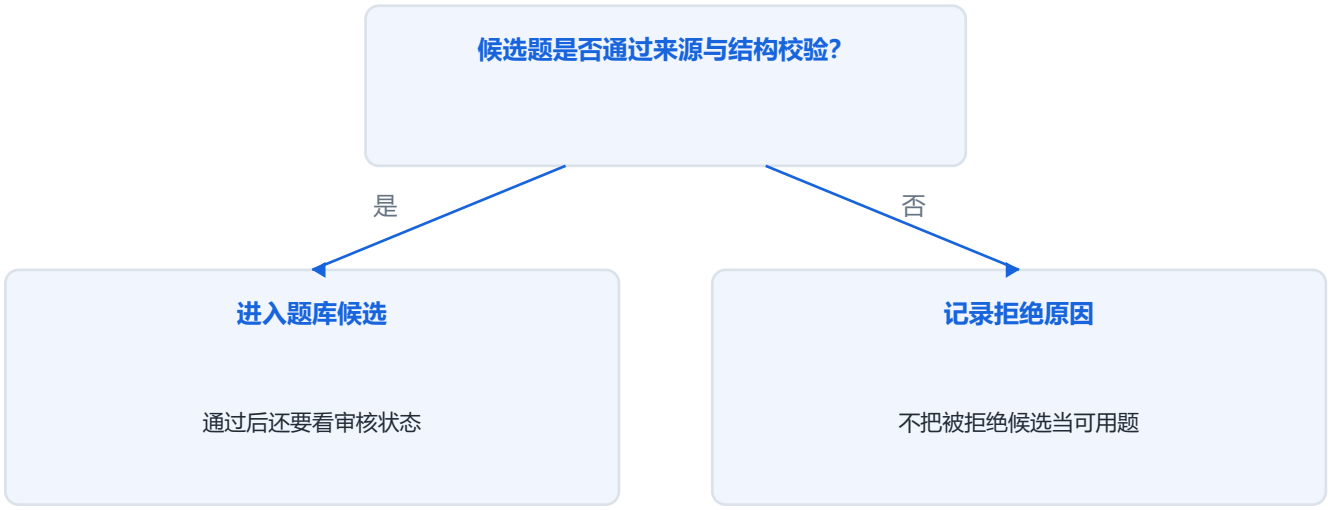


图 1-4 资料可用不等于题目合格；通过校验也不等于已审核发布。

后续章节会沿这些入口向下阅读。你不必一开始记住整个仓库，而应选择一个具体问题，例如“这道题为什么没有入库”，先看任务结果和质量门禁，再返回 Schema 与链。这样的阅读路线比从第一行顺序读完全部源码更容易形成因果关系。

1.3 命名先对齐：Model、Chain 与 Agent

Model（模型）是调用语言模型的对象，通常把消息转换成模型回复。Chain（链）把提示词、模型和解析器按固定步骤连接。Agent（智能体）则在受控循环中选择工具，工具结果再影响下一次模型调用。三个名称描述的层级不同，一个链或 Agent 都可以使用模型。

名词	本项目名称	怎样识别
Model / 模型	get_chat_model	按出题、答后讲解、辅导等用途配置模型
Chain / 固定链	create_knowledge_point_chain；create_question_generation_chain	程序固定安排提示、模型、解析步骤
Agent / 智能体	create_tutor_agent	模型在注册工具和预算中选择下一步
Workflow / 执行轨迹	WorkflowRunModel；WorkflowStepModel	记录运行过程；不是另一组智能体
Index / 索引	MilvusDocumentIndex；MilvusQuestionIndex	不同业务对象，不代表两种召回算法

表 1-2 名称用于解释职责，不依赖旧文件名猜测架构

例如“根据 TCP 教材生成 5 道选择题”会走知识点提取和出题链，程序知道先后顺序；“我最近为什么经常答错”则可能需要进度、错题、已作答上下文与资料检索工具，执行路径取决于提问和工具结果。这就是固定链与辅导 Agent 的具体差异。

不要根据名字补出未实现能力

当前产品是一个辅导 Agent，开发时协作的 subagent 不属于产品架构。MCP、多智能体协作、BM25 混合检索和 RAGAS 自动评测是可讨论的扩展，不能当作已有功能。

历史设计文件和旧物理表名可能仍包含 agent 字样。当前教学使用统一的 Workflow 命名，兼容 alias 保持旧调用可用。阅读时优先看对象做了什么、谁调用它、输入输出是什么，而不是因为看到 agent_runs 就认定它记录多个 Agent 的协商过程。

1.4 Context、State 与三种保存位置

Context 在项目中有两种常见语境。build_generation_context 返回供出题模型阅读的文本；TutorContext 是服务器传入的运行上下文，固定知识库和会话锚点。State 是图执行期间不断更新的共享字段。Context、State 与数据库业务事实应分别理解，不能看到同一个英文词就认为它们都是聊天历史。

概念	保存什么	使用场景与边界
TutorContext	会话、轮次、知识库、作答锚点	服务器确定；模型不能通过工具参数随意改范围
TutorState	消息、证据账本、标志、结构化草稿	工具与模型执行期间更新的图状态
业务事实 / PostgreSQL	材料、题目、作答、掌握度、已接受回复	业务事务确定哪些结果对用户正式有效

概念	保存什么	使用场景与边界
Snapshot / Redis	可从业务库重建的学习会话缓存	丢失后重建；不是图执行检查点
Checkpoint / PostgreSQL saver	图状态和执行位置等	业务接受指针选择下一轮允许使用的检查点

表 1-3 范围、运行状态、正式事实与缓存的区别



图 1-5 辅导状态进入正式结果的过程。图最新快照不自动等于业务已接受结果。

举例：图可能已经生成了回复并保存检查点，但网络或进程异常导致业务完成事务没有成功。下一轮如果盲目读取最新图状态，就可能把用户尚未接受的草稿当作正式历史。第07章会通过实验说明接受指针为什么必要；本章先记住“最新”和“已提交”是两个判断。

1.5 从目录到分层：函数为什么放在这里

位置	职责	阅读问题
apps/web/src/views	Vue 页面、交互和恢复提示	按钮提交了哪个 API？哪些信息仅用于展示？
api/routes 与 dependencies	请求模型、路由与服务装配	如何把 HTTP 输入传给业务服务？
application	业务用例与范围校验	步骤、错误、事务边界由谁决定？
domain/models 与 ports	领域数据与依赖接口	业务需要哪些字段和外部能力？
infrastructure 与 ingestion	数据库、缓存、存储、索引、解析	一个接口由哪种真实服务实现？
question_generation 与 agents/tutor	固定生成链与辅导工具循环	模型参与哪一步？程序还校验什么？

表 1-4 先读用例，再读依赖实现，避免陷入无上下文的底层细节

源码节选：注册业务路由

定位：apps/api/study_agent/api/router.py，第 1-19 行。

```python
from fastapi import APIRouter

from study_agent.api.routes import health, knowledge_bases, materials, questions, study, tutoring

api_router = APIRouter()
api_router.include_router(health.router)
api_router.include_router(knowledge_bases.router, prefix="/knowledge-bases", tags=["知识库"])
api_router.include_router(materials.knowledge_base_router, prefix="/knowledge-bases", tags=["资料"])
api_router.include_router(materials.material_router, prefix="/materials", tags=["资料"])
api_router.include_router(materials.job_router, prefix="/ingestion-jobs", tags=["资料处理任务"])
api_router.include_router(questions.material_router, prefix="/materials", tags=["题库生成"])
api_router.include_router(
    questions.job_router, prefix="/question-generation-jobs", tags=["题库生成"]
)
api_router.include_router(questions.router, prefix="/questions", tags=["题库"])
api_router.include_router(study.router, prefix="/study", tags=["学习"])
api_router.include_router(study.mastery_router, prefix="/mastery", tags=["掌握度"])
api_router.include_router(study.review_router, prefix="/reviews", tags=["复习"])
api_router.include_router(tutoring.router, tags=["学习辅导"])
```

`APIRouter` 把不同业务模块聚合成 API 入口；`prefix` 决定资源路径，`tags` 用于接口文档分组。这里仅做注册，不执行资料解析或模型推理。

随后 `create_app` 在外层加上配置的 api_prefix，默认是 /api。阅读 `/api/study/sessions` 时，应从 study 路由追到 StudyService，而不是从辅导 Agent 文件开始。

源码节选：领域中的资料片段

定位：apps/api/study_agent/domain/models.py，第 70-85 行。

```python
@dataclass(slots=True)
class DocumentChunk:
    id: UUID
    material_id: UUID
    chunk_index: int
    content: str
    token_count: int
    page_start: int | None
    page_end: int | None
    heading_path: list[str]
    content_hash: str
    vector_id: str | None
    embedding_model: str | None
    created_at: datetime | None = None

```

`DocumentChunk` 是资料片段的领域数据类。ID、材料归属、内容、页码和哈希用于后续持久化与引用；embedding_model 和 vector_id 记录索引关联。具体字符分块在第02章解释。

`@dataclass(slots=True)` 自动生成常用初始化方法并采用固定属性槽。它描述数据，不会自动调用 Embedding，也不等于数据库 ORM 表。

ports.py 使用 Protocol 描述仓库、对象存储等接口。例如 MaterialRepository.get 返回材料或 None，业务层可以依赖该契约，而不用把具体 SQL 写进每个用例。测试可以提供满足同一接口的显式假实现；正式环境则注入真实 PostgreSQL 等适配器。

读函数时记录三件事

记录输入、返回和副作用。例如“build_chunks 接收带 metadata 的文档，返回片段序列；不调用模型”；“StudyService.submit_answer 接收作答，保存业务事实并更新掌握度”。只记函数中文翻译不足以理解系统。

1.6 服务协作与失败定位

一个网页按钮的执行路径可能跨多个服务。FastAPI 处理请求；较长的资料处理和出题工作进入 Celery；Redis 提供队列与缓存；原文件保存到配置的对象存储后端；PostgreSQL 保存业务关系；Milvus 提供向量候选。语言模型和 Embedding 服务负责不同输入输出。



图 1-6 后台处理的阅读顺序。辅导请求有自己的同步执行与恢复机制，不把这张图套用到所有 API。

依赖	本项目用途	常见故障线索
PostgreSQL	业务记录、辅导审计和图保存	连接、约束、事务、接受指针
Redis	Celery broker/result 与学习快照	队列未消费、缓存过期；事实仍看业务库
本地存储 / MinIO	原始上传文件	存储 URI、文件权限、配置后端
Milvus	资料与题目向量索引	维度、集合、索引与候选归属
LLM / Embedding	生成回复 / 文本向量化	配置用途、超时、输出模式、模型维度

表 1-5 服务故障要对应到具体职责，不能统一解释为模型不聪明

比如页面长时间显示处理中，先查看 ingestion job 的 stage 和 worker 日志；如果向量候选返回跨知识库片段，查看检索二次核验；如果重复提交生成两份结果，查看幂等键与事务。错误从哪个阶段发生，决定你应该读哪一段源码。第02～07章会逐条展开这些检查。

1.7 首个可运行实验：只看路由和版本

这个实验不启动服务生命周期，不访问数据库，也不发送模型请求。它创建应用的接口描述并打印四个主要资源路径，同时显示当前安装版本。把完整代码保存成项目根目录下的临时 Python 文件，使用 apps/api/.venv/Scripts/python.exe 运行，工作目录为 D:/Study/StudyAgent。

完整可运行实验：检查真正存在的资源入口

```python
from importlib.metadata import version
from study_agent.config import get_settings
from study_agent.main import create_app

settings = get_settings()
paths = create_app().openapi()["paths"]
suffixes = ["/knowledge-bases", "/questions", "/study/sessions",
            "/tutor/conversations"]
for suffix in suffixes:
    path = settings.api_prefix + suffix
    print(path, sorted(paths[path]))
print("langchain:", version("langchain"))
print("langgraph:", version("langgraph"))
print("chunk_chars:", settings.chunk_size, settings.chunk_overlap)
```

`create_app()` 返回 FastAPI 应用；`openapi()` 生成接口结构字典，paths 是已注册路径集合。`settings.api_prefix` 防止把默认 /api 写成永远不变的事实。

`version()` 查询当前解释器实际安装的包版本。若结果与教材基准不同，先运行准备脚本检查环境，再讨论代码接口是否变化。不要复制或打印 .env 中的访问令牌与 API Key。

结果解释：默认前缀下可见 /api/knowledge-bases、/api/questions、/api/study/sessions、/api/tutor/conversations；版本为 LangChain 1.4.3、LangGraph 1.2.12，分块配置为 800/120。

实验验证的是“接口已注册、环境能导入、版本可识别”。它没有验证上传成功、向量检索效果或真实模型能力。这样的边界说明会贯穿所有章节：每个实验必须指出用了真实框架、固定数据还是外部供应商，避免把一次离线成功扩写成整个产品都已充分验收。

- 已安装环境：从项目根目录执行准备脚本，可加 -SkipInstall。
- 先跑 examples/learning/rag_context.py，再跑 agent_tool_loop.py，最后跑 langgraph_review.py。
- 脚本打印固定样本的结构与路径。日常重复实验不会隐式调用真实供应商。

1.8 学习规范与证据怎么读

之前的学习准备已经完成依赖对齐、命名纠正、实验补齐和验收记录。正式教材以当前工作区代码与锁定环境为准。第0章是准备执行前的样章，其中旧环境结论仅供理解当时检查过程。遇到历史方案与当前实现不一致时，应先看源码和最新准备说明。

证据类型	可以说明	不能据此推断
当前源码与离线实验	字段、调用顺序、固定案例输出	真实模型在任意问题上都正确
历史 149 项离线测试	当时的规则与回归路径通过	所有服务已经一起运行
历史 19 项 PostgreSQL 验收	隔离库的事务、锁、幂等和恢复	真实语言模型质量
历史供应商 2 个问题	固定 TCP 样本可完成调用和引用	大规模准确率或确定错因
历史 39 个浏览器案例	38 个 HTTP fixture 加 1 条服务链路	真实模型浏览器端到端质量

表 1-6 2026-10-07 学习准备记录的实际范围；本次制教材未重新调用供应商

本次 PDF 的完整可运行代码会在默认解释器重新执行，截图来自现有业务页面；少数交互演示使用真实 Vue 页面配合隔离 HTTP fixture，会在图下注明。这样既能展示答案反馈等状态，又不会为截图改变你的正式学习记录或产生额外模型费用。

阅读顺序与练习要求

每章先说场景和输入，再解释中英文名词，随后看图、表、源码节选和完整实验。节选用于定位，不宣称单独可运行；完整实验包含 imports 和明确结果。练习回答要同时说明行为与证据边界。

1.9 用一个问题串起八章

把整套课程想成追踪一段 TCP 讲义：第02章追踪“文字怎样变片段”，第03章追踪“问题怎样找到可信资料”，第04章追踪“资料怎样变成可审查题目”，第05章追踪“辅导怎样取证和回答”。同一资料经过不同路径，能帮助你发现共享数据与不同控制流程。



图 1-7 章节依赖路线：先建立资料证据，再理解生成、Agent 与持久化，最后回到学习结果。

第06章先用没有模型的独立 StateGraph 学习节点、条件边、reducer 和中断恢复，再映射到 create_agent 创建的生产图。这样你能分别观察图机制与模型选择。第07章进一步说明图保存不等于业务成功，第08章说明模型建议不等于确定性学习规则。

- 选一份小资料，只记录一个真实片段的 ID、页码和引用链。
- 选一次正常回复与一次证据不足回复，比较工具调用、证据和最终状态。
- 选一次响应丢失场景，画出客户端同 ID 重发与服务端重放结果。
- 完成后再讨论分批覆盖、混合检索或自定义图等扩展，不把扩展当作现状。

1.10 综合练习与参考思路

练习 1：用户问最近错题原因，与用户提交答案的职责有什么差别?

参考思路：提交答案走确定性判分、掌握度和复习更新；错题辅导读取业务事实并提供解释建议。辅导工具不自行修改这些成绩和计划。

练习 2：资料 ready 且向量索引存在，为什么仍可能没有足够题目?

参考思路：出题上下文有覆盖限制，模型候选可能不合格式或引用要求，质量门禁会拒绝候选；合格题仍受审核状态约束。

练习 3：如何判断一个离线 Agent 实验是否证明真实模型质量?

参考思路：先看模型和工具数据是否为 fixture。使用真实 create_agent 只证明框架与固定调用契约；质量需要独立样本与人工审阅。

练习 4：读源码时如何避免把缓存和检查点当作正式历史?

参考思路：先找业务事实保存位置与接受指针，再看快照是否可重建、检查点是否已业务提交。不能只因时间最新就接受全部字段。

1.11 源码阅读清单与下一步

文件或入口	关注的函数 / 类	阅读时回答的问题
README.md	项目说明	如何启动并理解当前实现范围
apps/api/study_agent/api/router.py	api_router	每条业务路径对应什么模块
apps/api/study_agent/main.py	create_app / lifespan	应用创建与资源启动为何不同
apps/api/study_agent/domain/models.py	DocumentChunk / Question / StudySession	领域数据的归属和关键字段
apps/api/study_agent/domain/ports.py	Repository Protocol	业务依赖哪种接口而非哪段 SQL
docs/技术方案/学习准备与章节计划.md	正式八章与验证边界	如何区分历史结果和当前执行

下一章：第02章从文件与 metadata 开始，逐步解释解析、字符分块、稳定 ID 和来源定位。

主要参考：本地飞书 LangChain、Agent、RAG 与 LangGraph 课程的教学组织方式，以及当前项目源码。具体框架接口以锁定环境与官方文档为准。